

Handout: 725103 Data Structure and Algorithm

สัปดาห์ที่ 3: Intro to Complexity Analysis (big-O notation)

ภาคปลาย ปีการศึกษา 2568 | อาจารย์ผู้สอน: ปพนสิร์ แยม์โชติ

เป้าหมายการเรียนรู้ประจำสัปดาห์ (Learning Outcomes)

เมื่อจบคาบนี้ นักศึกษาสามารถ...

- เข้าใจความหมายของ big-O ในการวิเคราะห์ความซับซ้อนของการทำงานของ program ผ่าน pseudocode ได้ (แยกได้ระหว่างความซับซ้อนและความเร็วในการรัน)
- สามารถนับจำนวนครั้งการดำเนินการของโปรแกรมจาก pseudocode เพื่อวิเคราะห์ความซับซ้อนของโปรแกรมได้

1 คณิตศาสตร์พื้นฐานสำหรับการวิเคราะห์ (Math Warm-up)

เราจะใช้คณิตศาสตร์แค่ ``ระดับจำเป็น" เพื่อคุยเรื่องการเติบโตของเวลา (ไม่เน้นพิสูจน์ยาก ๆ)

1.1 สรุปสัญลักษณ์และผลรวมที่ใช้บ่อย

- ผลรวม $1 + 2 + \dots + n = \frac{n(n+1)}{2} \approx n^2$
- ผลรวม $1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} \approx n^3$
- ถ้าลูปรทำงาน n รอบ และในแต่ละรอบทำงานคงที่ $\Rightarrow$ รวมเป็น ``ประมาณ cn " (โตตาม n)
- ถ้าลูปรซ้อนกัน (nested loop) เช่น ทำ n รอบ และข้างในทำ n รอบ $\Rightarrow$ รวมเป็น ``ประมาณ cn^2 "

1.2 แนวคิดของ $\log n$ แบบไม่ใช่แคลคูลัส

- $\log_2 n$ คือ ``จำนวนครั้งที่หาร 2 จนเหลือ 1" (ปัดขึ้น/ลงได้ตามบริบท)
- ตัวอย่าง: $n = 8$ หาร 2 สามครั้ง: $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ดังนั้น $\log_2 8 = 3$

2 Big-O และการนับจำนวน operation

2.1 ทำไมไม่พูดแค่ว่า ``เร็ว" หรือ ``ช้า"?

ความเร็วจริงขึ้นกับเครื่อง, ภาษา, ตัวแปรแวดล้อม แต่ Big-O สนใจ ``แนวโน้มเมื่อข้อมูลโตขึ้น" ดังนั้น Big-O เหมาะกับการเปรียบเทียบวิธีแบบเป็นระบบ (โดยไม่ต้องรันจริง)

อ่านแบบนักศึกษา (ไม่ต้องท่องนิยาม)

- $O(1)$ = ทำงาน ``คงที่" ไม่ขึ้นกับ n
- $O(n)$ = ทำงานเพิ่มตามขนาดข้อมูลแบบเส้นตรง
- $O(n^2)$ = โตเร็วขึ้นมาก (เช่น loop ซ้อน loop)
- $O(\log n)$ = โตช้า (เช่น ลดปัญหาครึ่งหนึ่งทุกครั้ง)

2.2 เราจะนับ operation อะไรบ้าง? (Cost Model)

ในวิชานี้ เราจะนับแบบง่าย (เพื่อให้ฝึกได้เร็ว) โดยถือว่าแต่ละอย่างด้านล่าง = 1 operation

- การกำหนดค่า (assignment): $x = \dots$
- การคำนวณเลขพื้นฐาน 1 ครั้ง: $+, -, *, /$
- การเปรียบเทียบ (comparison): $<, <=, ==, >=, >$

หมายเหตุ: การนับแบบนี้ไม่ใช่มาตรฐานเดียวในโลกจริง แต่เพียงพอสำหรับการเข้าใจ Big-O ในระดับพื้นฐาน

2.3 ตัวอย่าง 1: การบวกเลข 1 ถึง n

Algorithm 1 SumToN (นับ operation)

Require: n คือจำนวนเต็มบวก

```
1:  $sum = 0$ 
2: for  $i = 1$  to  $n$  do
3:    $sum = sum + i$ 
4: end for
5: return  $sum$ 
```

การนับแบบคร่าว ๆ ดังนั้น $T(n) \approx$ _____

Exercise 1. ถ้าไม่ใช้วิธีการนับละเอียด อะไรในอัลกอริทึมนี้ที่บอกไปว่าเป็น $O(n)$

2.4 ตัวอย่าง 2: loop ซ้อน loop

Algorithm 2 CountPairs

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $n$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

แนวความคิดการนับ ดังนั้น $T(n) \approx$ _____

Exercise 2. ถ้าเปลี่ยนลูปด้านในให้วิ่งแค่ถึง $j = i$ (คือ **for** $j = 1$ to i) จะเหลือประมาณกี่ครั้ง? และเป็น $O(\cdot)$ อะไร?

Algorithm 3 CountPairs

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $i$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

2.5 ตัวอย่าง 3: ลดปัญหาครึ่งหนึ่งทุกครั้ง

Algorithm 4 HalfUntilOne

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = \lfloor n/2 \rfloor$ 
4:    $count = count + 1$ 
5: end while
6: return  $count$ 
```

คำถาม: ลูปทำกี่รอบ?

ดังนั้น $T(n) \approx$ _____

Exercise 3. ให้ $n = 20$ จง trace ค่า n ในแต่ละรอบ จนลูปหยุด และเขียนจำนวนรอบทั้งหมด

Algorithm 5 Legendre Formula: ไว้ใช้หาว่า $n!$ ลงท้ายด้วย 0 กี่ตัว

Require: n คือจำนวนเต็มบวก

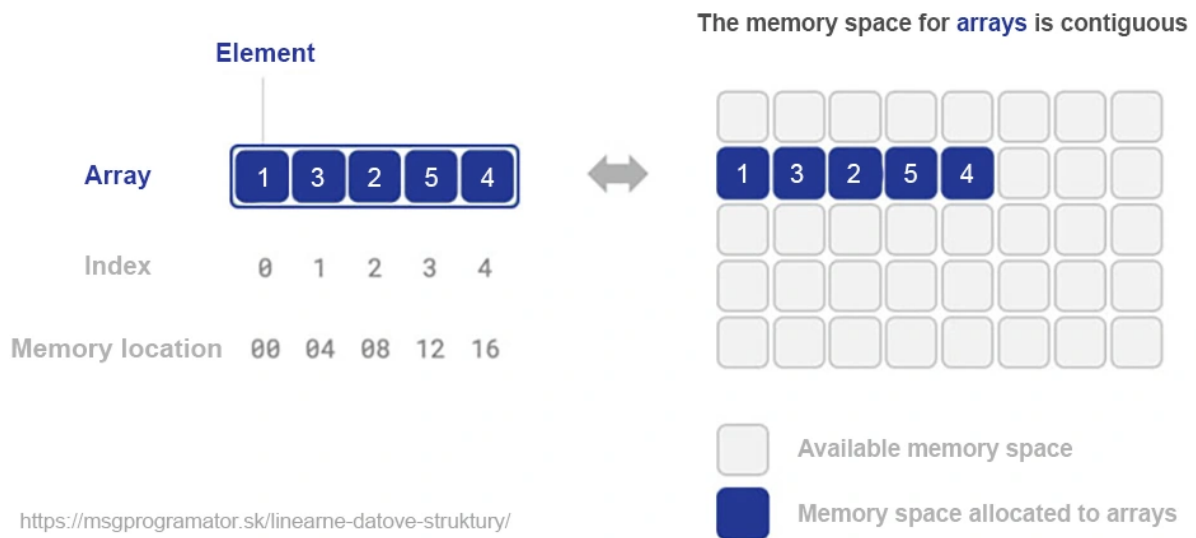
```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = \lfloor n/5 \rfloor$ 
4:    $count = count + n$ 
5: end while
6: return  $count$ 
```

ดังนั้น $T(n) \approx$ _____

Exercise 4. ให้ $n = 1000$ จง trace ค่า n ในแต่ละรอบ จนลูปหยุด และเขียนจำนวนรอบทั้งหมด

2.6 ตัวอย่าง 4: เปรียบเทียบกับ best/worst case

2.6.1 ทบทวน(แนะนำ) array เบื้องต้น



ในการเขียน pseudocode เราจะเขียน $A[0..(n-1)]$ แทนอาร์เรย์ที่มีสมาชิก n ตัวคือสมาชิกที่อยู่ดัชนี (index) ที่ 0 ถึง index ที่ $n-1$ และจะมีการประกาศว่าเป็น array ที่ไว้เก็บข้อมูลประเภทไหนและจะเก็บแต่ข้อมูลประเภทนั้นอย่างเดียว (ถึงแม้ใน Python เราจะใช้ list ที่สามารถเก็บข้อมูลผสมประเภทได้)

และเมื่อเราจะเข้าถึงสมาชิกที่ index ใดๆ ก็ตาม จะใช้การเขียน $A[i]$ แทนการเข้าถึงข้อมูลที่อยู่ดัชนี i (ดังนั้นเราจะเข้าถึงได้ตั้งแต่ $A[0], \dots, A[n-1]$) โดยตัวอย่างการใช้งานดังนี้

- $x = A[i]$ คือ _____
- $A[i] = x$ คือ _____
- $A[i] == x$ คือ _____
- $A[i] = A[j]$ คือ _____
- $A[i] == A[j]$ คือ _____
- $A[i] < A[j]$ คือ _____

2.6.2 ตัวอย่างโปรแกรมที่ทำกับ array

Algorithm 6 FindFirstZero

Require: $A[0..(n-1)]$ คืออาร์เรย์ของจำนวนเต็ม

```
1: for  $i = 0$  to  $n - 1$  do
2:   if  $A[i] == 0$  then
3:     return  $i$ 
4:   end if
5: end for
6: return  $-1$ 
```

▷ ไม่พบ

- **Best case:** เจอศูนย์ตั้งแต่ตัวแรก ($i = 1$) $\Rightarrow$ _____
- **Worst case:** ไม่มีศูนย์เลย หรืออยู่ตัวสุดท้าย $\Rightarrow$ _____

Exercise 5. ถ้าในอาร์เรย์มีศูนย์อยู่เสมอ ``เฉลี่ย" จะเป็น $O(\cdot)$ อะไร? กรณีดีที่สุด O เท่าไหร่ และกรณีแย่สุด O เท่าไหร่

3 แบบฝึกในคาบ: จากการนับ operation $\rightarrow$ สรุปเป็น Big-O

โจทย์ A (Warm-up)

Algorithm 7 A

Require: n คือจำนวนเต็มบวก

```
1:  $x = 0$ 
2: for  $i = 1$  to  $n$  do
3:    $x = x + 2$ 
4: end for
5: return  $x$ 
```

Exercise 6. (1) x จะได้ค่าเท่าไรเมื่อจบ? (2) อัลกอริทึมนี้เป็น $O(\cdot)$ อะไร?

โจทย์ B (นับรอบไม่เท่ากัน)

Algorithm 8 B

Require: n คือจำนวนเต็มบวก

```
1:  $count = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $2n$  do
4:      $count = count + 1$ 
5:   end for
6: end for
7: return  $count$ 
```

Exercise 7. จงนับจำนวนครั้งที่บรรทัดในสุดทำงาน (ในรูปของ n) และสรุปเป็น $O(\cdot)$

โจทย์ C (รูปเพิ่มทีละ 2)

Algorithm 9 C

Require: n คือจำนวนเต็มบวก

```
1:  $i = 1$ 
2: while  $i \leq n$  do
3:    $i = i + 2$ 
4: end while
```

Exercise 8. รูปนี้ทำงานกี่รอบโดยประมาณ? และเป็น $O(\cdot)$ อะไร?

โจทย์ D (สามเหลี่ยม)

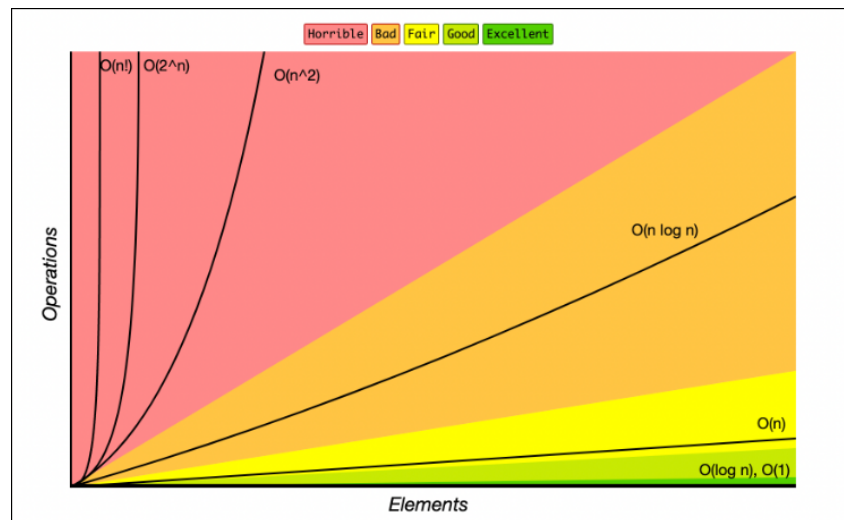
Algorithm 10 D

Require: n คือจำนวนเต็มบวก

```
1:  $s = 0$ 
2: for  $i = 1$  to  $n$  do
3:   for  $j = 1$  to  $i$  do
4:      $s = s + 1$ 
5:   end for
6: end for
7: return  $s$ 
```

Exercise 9. (1) บรรทัดในสุดทำทั้งหมดกี่ครั้ง (เขียนเป็นผลรวม) (2) สรุปเป็น $O(\cdot)$

3.1 สรุปกราฟการเติบโตที่ควรรู้จัก



รูปแบบ	มาจากอะไรบ่อย	ตัวอย่างแนวคิด
$O(1)$	ทำงานคงที่	เข้าถึง $A[i]$ 1 ครั้ง
$O(\log n)$	ลดครึ่งทุกครั้ง	binary search, ทหาร 2 জনমত
$O(n)$	ทำทีละตัวครบทุกตัว	เดินดูทุกคนในคิว
$O(n \log n)$	แบ่งครึ่ง + ทำซ้ำหลายชั้น	divide-and-conquer หลายแบบ
$O(n^2)$	loop ซ้อน loop	เปรียบเทียบทุกคู่
$O(2^n), O(n!)$	มีจำนวนงานย่อยแทบไม่ค่อยลดลง (เช่น อันตราย)	เล่น Tower of Hanoi

4 การบ้านหลังเรียน

คำชี้แจง

อัลกอริทึมต่อไปนี้เป็นอัลกอริทึมเดียวกันกับการบ้านของสัปดาห์ที่ 2 ที่นักศึกษาได้ trace การทำงานของอัลกอริทึมไปแล้ว ในสัปดาห์นี้ให้นักศึกษาพิจารณาความซับซ้อนของอัลกอริทึมด้วยวิธีการนับจำนวนครั้งการดำเนินการ

ข้อ 1: if/else (เลือกทางเดิน)

Algorithm 11 HW1

Require: a, b ที่เป็นจำนวนเต็ม

```
1: if  $a > b$  then
2:    $m = a$ 
3: else
4:    $m = b$ 
5: end if
6:  $m = m - 3$ 
7: return  $m$ 
```

ข้อ 2: for (ทำซ้ำจำนวนรอบที่รู้ล่วงหน้า)

Algorithm 12 HW2

Require: n เป็นจำนวนเต็ม

```
1:  $ans = 1$ 
2: for  $i = 1$  to  $n$  do
3:    $ans = ans * i$ 
4: end for
5: return  $ans$ 
```

ข้อ 3: while (ทำซ้ำจนกว่าจะหยุด) (1)

Algorithm 13 HW3

Require: n เป็นจำนวนเต็ม

```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = n - 3$ 
4:    $count = count + 1$ 
5: end while
6: return  $count$ 
```

ข้อ 4: while (ทำซ้ำจนกว่าจะหยุด) (2)

Algorithm 14 HW3

Require: n เป็นจำนวนเต็ม

```
1:  $count = 0$ 
2: while  $n > 1$  do
3:    $n = n/2$                                 ▷ สมมติว่าหารแล้วปัดเศษลงเสมอ (เพราะเก็บเป็น int)
4:    $count = count + 1$ 
5: end while
6: return  $count$ 
```

ข้อ 5: ผสม for + if (ฝึกคิดเงื่อนไขในรูป)

Algorithm 15 HW5

```
1:  $score = 0$ 
2: for  $i = 1$  to 6 do
3:   if  $i$  is even then                        ▷  $i$  เป็นเลขคู่
4:      $score = score + 2$ 
5:   else
6:      $score = score + 1$ 
7:   end if
8: end for
9: return  $score$ 
```
